



Entrega 3: Análisis Comparativo de Rendimiento en Implementaciones de Tablas Hash

Universidad de Concepción

Facultad de Ingeniería, Ingeniería Civil Informática

Estructura de datos

Docente: José Sebastián Fuentes Sepúlveda

Integrantes:

**Jhostian Matías San Martín Morales
Felipe Alejandro Tilleria Morales**

8 de julio 2026

Introducción.

En el procesamiento de grandes volúmenes de datos, como en este caso, el conteo de frecuencias es fundamental para un futuro análisis, aunque también es computacionalmente exigente. La eficiencia con la que se almacenan y recuperan estos datos es fundamental para determinar la viabilidad técnica de un sistema de análisis tan masivo, y por lo tanto, el seleccionar una estructura de datos adecuada es clave para una buena implementación, ya que influye directamente en la latencia y en la saturación de los recursos del hardware.

Ya en este contexto, el objetivo de este proyecto consiste en poder cuantificar la cantidad de publicaciones por usuarios, dentro del dataset que contiene los tweets relacionados a las elecciones australianas de 2019 (el cual comprende aproximadamente 130000 tweets utilizables), y utilizar diferentes estructuras para comparar la efectividad de cada una al contabilizar. Para lograrlo, se utilizaron tanto estructuras como la estándar de C++ (`std::unordered_map`), así como implementaciones personalizadas, tanto de hashing abierto como de hashing cerrado, con múltiples estrategias de colisión. En cuanto al procesamiento de los datos, se realizó mediante un parser optimizado para la lectura de archivos CSVs.

En cuanto al análisis de los resultados, se realizaron importaciones de los CSVs con los resultados de los tiempos de ejecución a una hoja de cálculo de google, donde se crearon tablas y gráficos correspondientes. Y, como conclusión preliminar, se puede afirmar que los tiempos varían enormemente dependiendo de la estructura y la clave escogida, pero esto se verá en profundidad más adelante.

Descripción de las estructuras de datos comparadas.

El rendimiento de las tablas hash no está únicamente en su capacidad de almacenamiento, sino que la calidad de su función hash y su estrategia de manejo de colisiones afectan de manera significativa a su rendimiento, ya que, una distribución deficiente de claves puede degradar una búsqueda de tiempo constante $O(1)$ a una búsqueda lineal $O(n)$, lo que cargaría al sistema de manera innecesaria. A continuación se realizará una breve descripción de las estructuras utilizadas para la comparación:

Hashing Abierto (Encadenamiento)

En esta estructura cada bucket (en nuestra implementación se usa una capacidad fija de 200003 buckets, ya que es un primo) almacena una lista de colisiones por lo que insertar y buscar tienen costo promedio $O(1)$ y peor caso $O(n)$ si es que muchas caen en el mismo bucket, por lo que una buena función hash es necesaria, en nuestro caso se utilizó $h(k) = k \bmod m$ para *user_id* y para *user_screen_name* se usó un hash polinomial con base 31 $\bmod m$, que toma en cuenta cada uno de los caracteres del nombre de usuario, para asegurarse que nombres similares entreguen valores distintos.

Hashing Cerrado con linear probing

Aquí todas las claves se guardan dentro del arreglo sin externalidades, y las colisiones se manejan secuencialmente probando posiciones tal que $(h(k) + i) \bmod m$ esté libre. Tanto inserción como búsqueda son $O(1)$ en promedio con factor de carga bajo y se torna hasta su peor caso en $O(n)$ a medida que el factor sube, aquí también se utilizaron 200003 buckets y las mismas dos funciones hash dependiendo de la clave utilizada.

Hashing Cerrado con quadratic probing

Tanto buckets como funciones hash idénticas al anterior, solo que, este resuelve colisiones con saltos cuadráticos con $(h(k) + i^2) \bmod m$ reduciendo un poco el clustering de linear probing. Su promedio se mantiene en $O(1)$ y su peor caso en $O(n)$ tanto para búsquedas como para inserciones.

Hashing Cerrado con double hashing

Esta estructura resuelve las colisiones con una segunda función, para *user_id*: $1 + (k \bmod (m - 1))$ y para *user_screen_name* se usó un hash polinomial con base 37 $\bmod (m - 1)$, que toma en cuenta cada uno de los caracteres del nombre de usuario para asegurarse que nombres de usuario similares resulten en valores distintos. Todo lo demás es igual a las anteriores.

STL de C++ (unordered_map)

Esta es una tabla hash genérica de la biblioteca estándar con complejidades promedios $O(1)$ y peor caso $O(n)$ internamente usa buckets y rehash automáticos según el factor de carga.

Descripción del dataset y del ambiente de experimentación.

Se utilizó el dataset entregado (auspol2019) en formato CSV que contiene publicaciones de twitter relacionadas a las elecciones australianas en 2019. El archivo fue procesado a través de un parser personalizado que de manera secuencial fue leyendo línea por línea y extrayendo únicamente las columnas necesarias: *user_id* y *user_screen_name*.

El parser maneja las comillas para no cortar incorrectamente campos que contienen comas internas. Se encarga de descartar la cabecera y luego se separan los campos. Si la fila no contiene la cantidad mínima de columnas esperada o presenta errores de conversión en la clave, dicha fila se omite para mantener el proceso consistente y evitar que algunos datos malformados distorsionen los resultados. Por esto mismo es que en nuestra comparación los tweets rescatados del datasets son únicamente 130000 de los 180000 originales, así que no fue necesario modificar el archivo fuente original.

Los experimentos se ejecutaron en un único equipo manteniendo el mismo entorno para todas las estructuras y claves evaluadas:

- Nombre del SO: Microsoft Windows 11 Pro
- Procesador: AMD Ryzen 7 3700X 8-Core Processor, 3600 Mhz,
8 procesadores principales, 16 procesadores lógicos
- Producto placa base: TUF GAMING B550M-PLUS (WI-FI)
- Memoria física instalada (RAM): 32,0 GB
- Cache del procesador: L1 512 KB, L2 4096 KB (4 MB) y L3 32768 KB (32 MB)

Resultados experimentales.

Se evaluó el rendimiento en la construcción de las estructuras hash para contar tweets por usuario, usando dos claves dentro de los valores en el dataset: *user_id* y *user_screen_name*. Las mediciones se realizaron cada 10000 registros hasta el tamaño final (130449 registros que se extrajeron correctamente), con 20 repeticiones por cada configuración que dio origen a la tabla resultados que debido a su extensión está en esta [hoja de cálculos de google](#).

Sobre estos resultados se calculó por clave, estructura y tamaño, tanto el tiempo promedio en microsegundos como su desviación estándar en microsegundos. Todo esto para cada uno de las 20 repeticiones, y se almacenaron en la tabla métricas de la misma [hoja de cálculo](#).

La escalabilidad temporal para *user_id* se muestra en la Figura 1, y el mismo criterio para *user_screen_name* se muestra en la Figura 2. En ambas claves, las implementaciones de hashing cerrado (con sus 3 variantes) muestran mejores tiempos que el hashing abierto, sobre todo en tamaños más grandes. En cuanto al *unordered_map* implementado por la biblioteca general de C++, se comportó de mejor manera que las otras estructuras en la Figura 1 con la clave *user_id* y de peor manera en la Figura 2 con la clave *user_screen_name*. Así El mejor tiempo que se obtuvo de las comparaciones fue:

- *user_id*, utilizando hash cerrado lineal alcanzó un promedio de 1329.90 milisegundos al momento de leer todos los datos.
- *user_screen_name*, utilizando STL *unordered_Map* alcanzó un promedio de 8228.65 milisegundo al momento de leer todos los datos.

Respecto a la diferencia en cuanto a eficiencia por clave usada, la evidencia muestra que *user_id* es más eficiente que *user_screen_name* en todas las estructuras comprobadas, como se puede apreciar en la Figura 3. Esto se debe al menor costo de hashing y comparación de enteros del id frente a los strings que suponían usar el nombre del usuario como clave.

Por último, en cuanto a la desviación estándar, esta se mantiene relativamente acotada en *user_id* y llega a aumentar significativamente para *user_screen_name*, especialmente en tamaños de tabla más grande como se puede apreciar en las dos primeras figuras. Lo que es esperable por como se mencionó antes, el mayor costo de procesamiento para strings y su mayor sensibilidad a colisiones

Figura 1

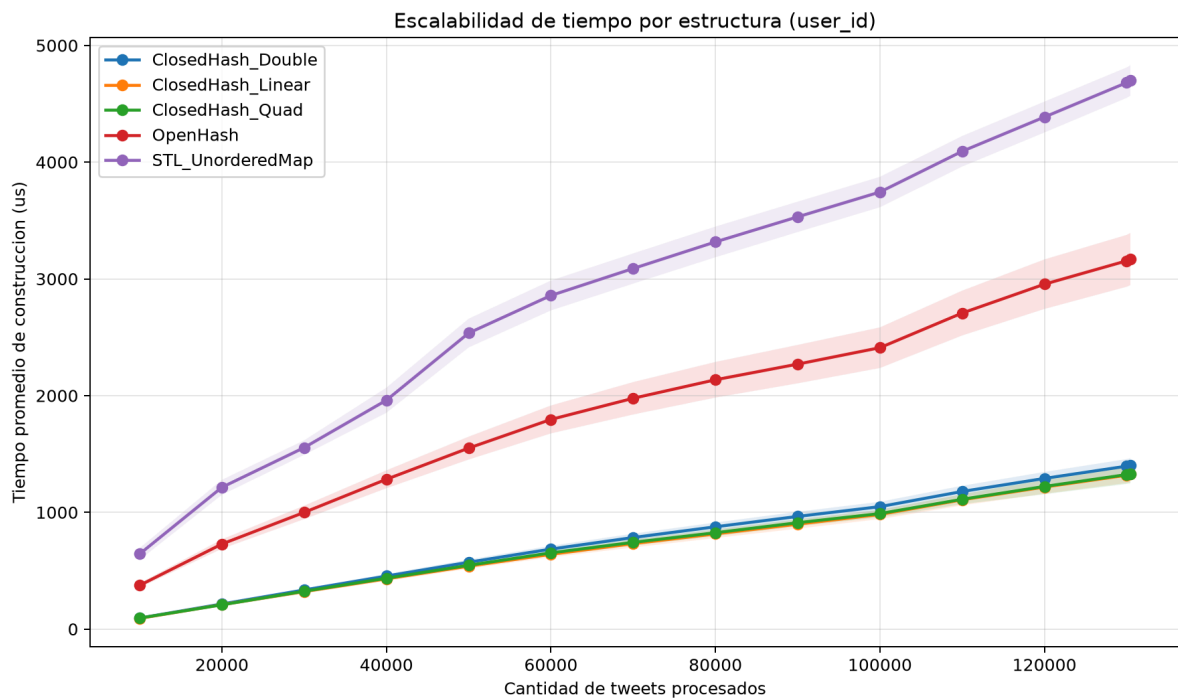


Figura 2

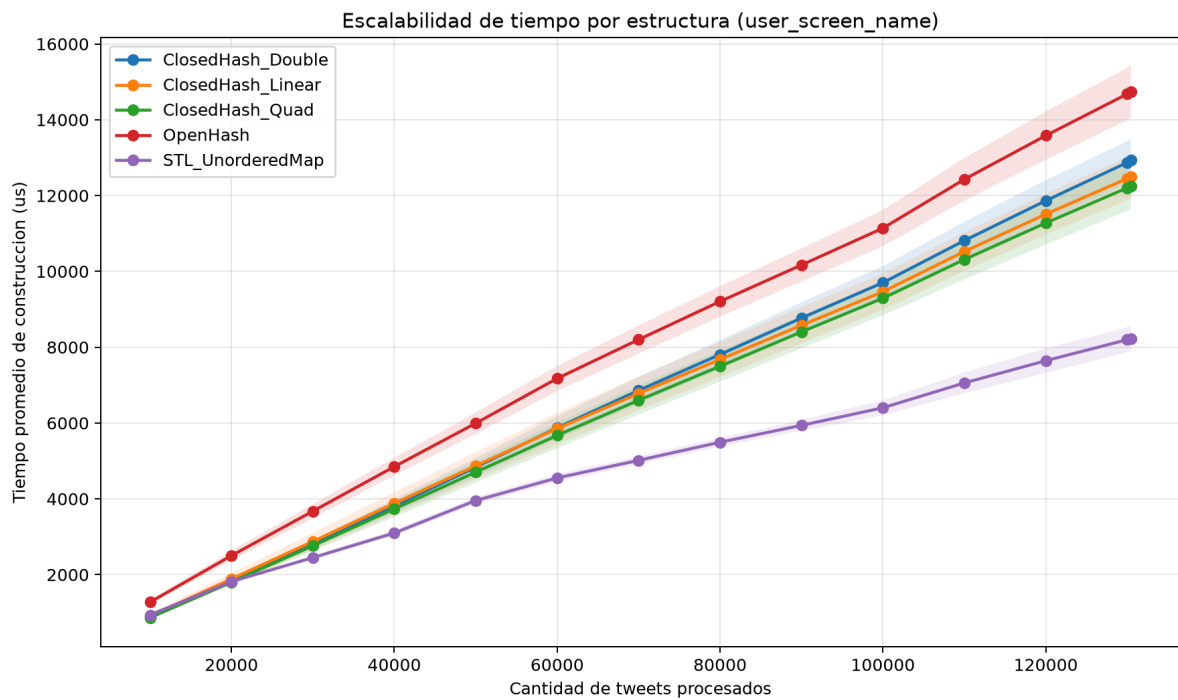
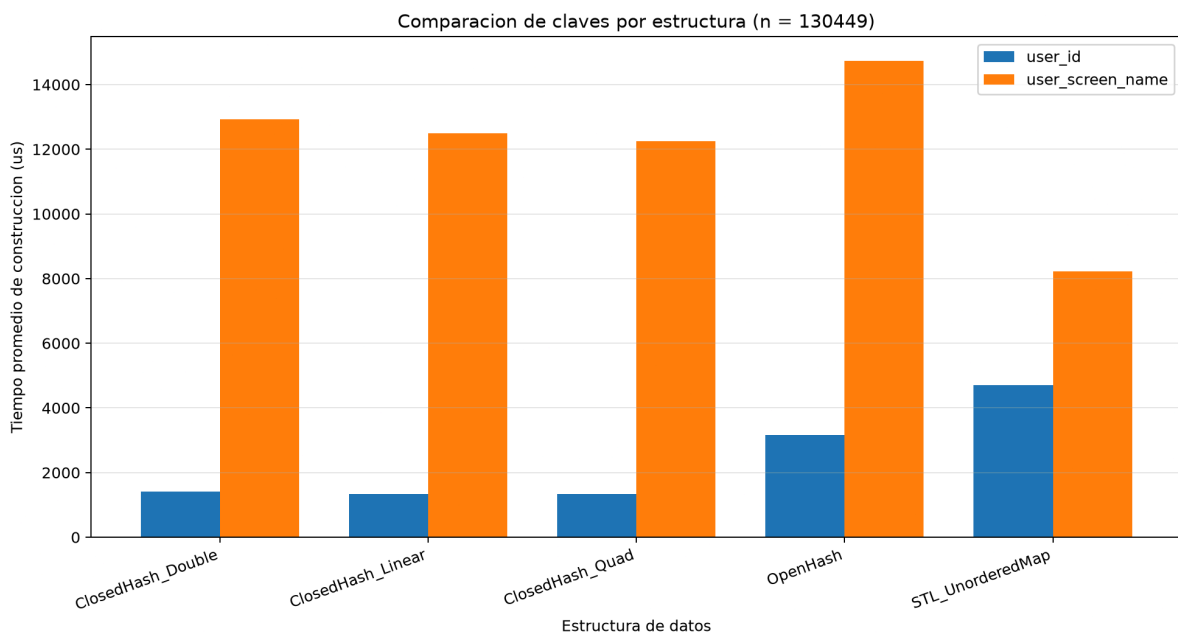


Figura 3



- En la Figura 1 y Figura 2, el sombreado alrededor de cada estructura es su desviación estándar

Conclusiones.

El análisis experimental de la contabilización sobre los tweets, entregó datos que nos permiten medir el impacto respecto al tipo de dato que se usa para la clave y la gestión de memoria en el rendimiento computacional.

El factor más determinante para el rendimiento del sistema, fue el tipo de clave elegido. Las mediciones demostraron una eficiencia claramente superior al usar 'user_id' sobre 'user_screen_name'. Esto puede deberse principalmente a que el primero es un entero, mientras que el segundo es una cadena de caracteres, lo que repercutió, tanto en la dificultad de lectura de las claves, como en la forma de funciones hash usadas para cada clave, ya que, mientras que para 'user_id', la función se resuelve mediante una sola operación simple, para 'user_screen_name' se itera sobre cada carácter del nombre. Además, a esto se le suma que, al resolver las colisiones, verificar la igualdad entre enteros, es computacionalmente insignificante si lo comparamos a evaluar la igualdad entre cadenas de caracteres.

A través de los resultados, se puede ver que las consultas con claves numéricas tardaron mucho menos en procesarse que las de clave de texto, pudiendo ver que las consultas sobre 'user_screen_name' resultaron ser más de cuatro veces más lentas.

Para 'user_id', el hashing cerrado con linear probing, resultó ser la estructura más rápida, seguida muy de cerca por el hashing cerrado con quadratic probing. Por otro lado, para 'user_screen_name', la estructura estándar de C++ (std::unordered_map), resultó ser la estructura más rápida, lo que debe ser resultado de que esta clave es una cadena de texto y la biblioteca estándar de C++ debe incluir optimizaciones para el manejo de estos casos más complejos. Por parte de nuestras implementaciones, el hashing cerrado con quadratic probing fue la que mejor se desempeñó, seguida de cerca por el linear probing.

Finalmente, podemos decir que para conjuntos de datos masivos como estos, el realizar búsquedas sobre claves más simples como lo pueden ser las compuestas por números enteros, es, no sólo más eficiente, si no que necesario. También, podemos destacar que, aunque el hashing abierto es más teóricamente sencillo y robusto, las limitaciones del hardware respecto al uso de punteros y memoria dinámica para su implementación, terminan por empeorar su rendimiento en entornos como este.